UGUI 事件系统（EventSystem）

事件系统是什么

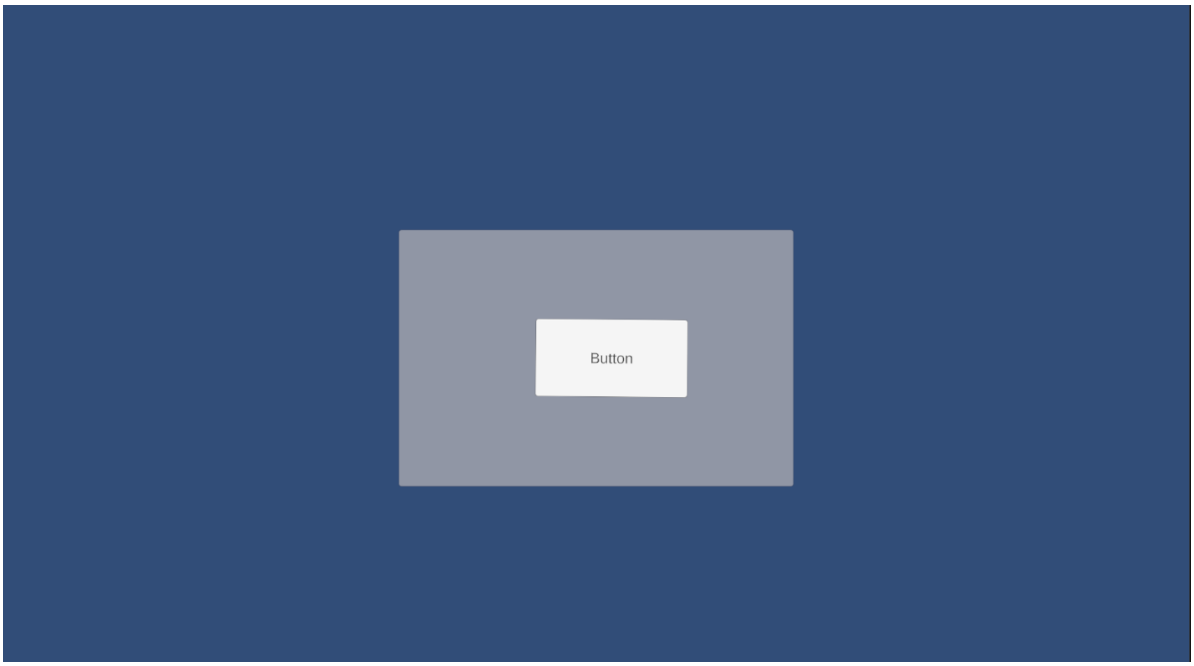
事件系统是一种根据输入（无论是键盘、鼠标、触摸还是自定义输入）向应用程序中的对象发送事件的方法。

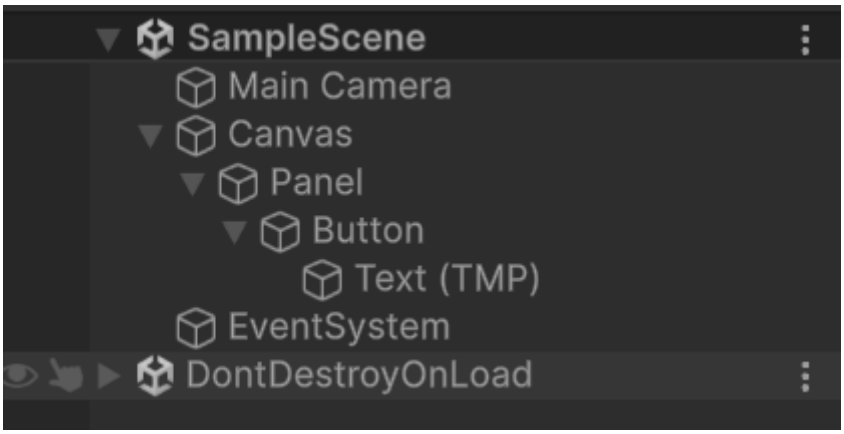
事件系统由几个组件组成，这些组件协同工作以发送事件。

事件系统的主要功能如下：

- 管理哪个游戏对象被视为已选中
- 管理当前使用的输入模块
- 管理光线投射（如有需要）
- 根据需要更新所有输入模块

实际看看一次点击按钮发生了什么






```
using UnityEngine;
using UnityEngine.UI;

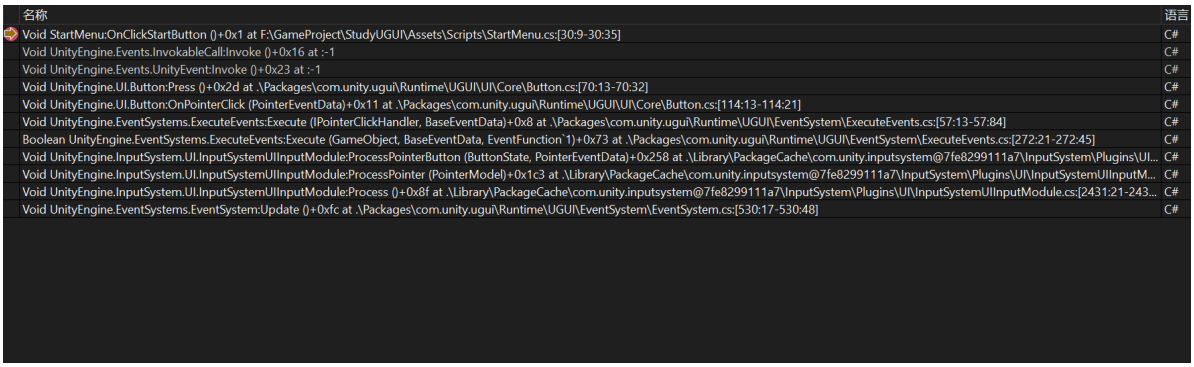
public class StartMenu : MonoBehaviour
{
```

```
    public Button startButton;

    void Start()
    {
        Debug.Log("StartMenu Start");
        startButton.onClick.AddListener(OnClickStartButton);
    }

    void OnClickStartButton()
    {
        Debug.Log("click button");
    }
}
```

函数调用路径：



- EventSystem:Update

  这里进行的是每帧一次的更新，没啥需要注意的，注意一下同时只Process一个输入模块即可

  EventSystem继承UIBehaviour，UIBehaviour继承MonoBehaviour

  这里的m_CurrentInputModule = "EventSystem (UnityEngine.InputSystem.UI.InputSystemUIInputModule)"

```
        protected virtual void Update()
        {
            if (current != this) //同一时间只有一个EventSystem更新
                return;
            TickModules(); //更新所有输入模块

            //判断当前使用输入模块是否变化
            bool changedModule = false;
            var systemInputModulesCount = m_SystemInputModules.Count;
            for (var i = 0; i < systemInputModulesCount; i++)
            {
                var module = m_SystemInputModules[i];
                if (module.IsModuleSupported() && module.ShouldActivateModule())
                {
                    if (m_CurrentInputModule != module)
                    {
                        ChangeEventModule(module);
                        changedModule = true;
                    }
                    break;
                }
            }
```

```
            // no event module set... set the first valid one...
            if (m_CurrentInputModule == null)
            {
                for (var i = 0; i < systemInputModulesCount; i++)
                {
                    var module = m_SystemInputModules[i];
                    if (module.IsModuleSupported())
                    {
                        ChangeEventModule(module);
                        changedModule = true;
                        break;
                    }
                }
            }

            if (!changedModule && m_CurrentInputModule != null)
                m_CurrentInputModule.Process();
        }
```

- InputSystemUIInputModule:Process

  PointerModel: 鼠标、一个触摸点状态

  处理一下pointmodel，也没啥特别的

```
   internal InlinedArray<PointerModel> m_PointerStates;
    public override void Process()
   {
       .......
       else
       {
           // Navigation input.
           ProcessNavigation(ref m_NavigationState);

           FilterPointerStatesByType();

           // Pointer input.
           for (var i = 0; i < m_PointerStates.length; i++)
           {
               ref var state = ref GetPointerStateForIndex(i);

               ProcessPointer(ref state);

               // If it's a touch and the touch has ended, release the pointer
state.
               // NOTE: We defer this by one frame such that OnPointerUp happens
in the frame of release
               //       and OnPointerExit happens one frame later. This is so
that IsPointerOverGameObject()
               //       stays true for the touch in the frame of release (see
UI_TouchPointersAreKeptForOneFrameAfterRelease).
               if (state.pointerType == UIPointerType.Touch &&
!state.leftButton.isPressed && !state.leftButton.wasReleasedThisFrame)
               {
                   RemovePointerAtIndex(i);
                   --i;
                   continue;
               }
```

```
                state.OnFrameFinished();
            }
        }
    }
```

- InputSystemUIInputModule:ProcessPointer

  在这里开始正式的处理，获得pointmodel所在物体，调用移动处理、按键处理

```
    // Mouse, pen, touch, and tracked device pointer input all go through here.
    private void ProcessPointer(ref PointerModel state)
    {
        var eventData = state.eventData;

        ........

        // Raycast from current position.
        // 射线检测获得当前鼠标所在的物体
        eventData.pointerCurrentRaycast = PerformRaycast(eventData);

        .......

        ////REVIEW: for touch, we only need the left button; should we skip
right and middle button processing? then we also don't need to copy to/from the
event

        // Left mouse button. Movement and scrolling is processed with event set
left button.
        eventData.button = PointerEventData.InputButton.Left;
        state.leftButton.CopyPressStateTo(eventData);

        // Unlike StandaloneInputModule, we process moves before processing
buttons. This way
        // UI elements get pointer enters/exits before they get button ups/downs
and clicks.
        ProcessPointerMovement(ref state, eventData);

        // We always need to process move-related events in order to get
PointerEnter and Exit events
        // when we change UI state (e.g. show/hide objects) without moving the
pointer. This unfortunately
        // also means that we will invariably raycast on every update.
        // However, after that, early out at this point when there's no changes
to the pointer state (except
        // for tracked pointers as the tracking origin may have moved).
        if (!state.changedThisFrame && (xrTrackingOrigin == null ||
state.pointerType != UIPointerType.Tracked))
            return;

        ProcessPointerButton(ref state.leftButton, eventData);
        ProcessPointerButtonDrag(ref state.leftButton, eventData);
        ProcessPointerScroll(ref state, eventData);

        // Right mouse button.
        eventData.button = PointerEventData.InputButton.Right;
        state.rightButton.CopyPressStateTo(eventData);
```

```
        ProcessPointerButton(ref state.rightButton, eventData);
        ProcessPointerButtonDrag(ref state.rightButton, eventData);

        // Middle mouse button.
        eventData.button = PointerEventData.InputButton.Middle;
        state.middleButton.CopyPressStateTo(eventData);

        ProcessPointerButton(ref state.middleButton, eventData);
        ProcessPointerButtonDrag(ref state.middleButton, eventData);
    }
```

- InputSystemUIInputModule:ProcessPointerButton

  处理按键事件，在这里可以看到Drag，pointup，pointdown，Click。。。等事件在这里发送出去

```
        private void ProcessPointerButton(ref PointerModel.ButtonState button,
PointerEventData eventData)
        {
            // 这里点击时GO实际上是TMP!
            var currentOverGo = eventData.pointerCurrentRaycast.gameObject;

            if (currentOverGo != null &&
PointerShouldIgnoreTransform(currentOverGo.transform))
                return;

            // Button press.
            if (button.wasPressedThisFrame)
            {
                button.pressTime = InputRuntime.s_Instance.unscaledGameTime;

                eventData.delta = Vector2.zero;
                eventData.dragging = false;
                eventData.pressPosition = eventData.position;
                eventData.pointerPressRaycast = eventData.pointerCurrentRaycast;
                eventData.eligibleForClick = true;
                eventData.useDragThreshold = true;

                var selectHandler =
ExecuteEvents.GetEventHandler<ISelectHandler>(currentOverGo);

                // If we have clicked something new, deselect the old thing and
leave 'selection handling' up
                // to the press event (except if there's none and we're told to
not deselect in that case).
                if (selectHandler != eventSystem.currentSelectedGameObject &&
(selectHandler != null || m_DeselectOnBackgroundClick))
                    eventSystem.SetSelectedGameObject(null, eventData);

                // Invoke OnPointerDown, if present.
                var newPressed = ExecuteEvents.ExecuteHierarchy(currentOverGo,
eventData, ExecuteEvents.pointerDownHandler);

                var pointerClickHandler =
ExecuteEvents.GetEventHandler<IPointerClickHandler>(currentOverGo);
```

```
                // If no GO responded to OnPointerDown, look for one that
responds to OnPointerClick.
                // NOTE: This only looks up the handler. We don't invoke
OnPointerClick here.
                if (newPressed == null)
                    newPressed = pointerClickHandler;

                // Reset click state if delay to last release was too long or if
we didn't
                // press on the same object as last time. The latter part we
don't know until
                // we've actually run the press handler.
                button.clickedOnSameGameObject = newPressed ==
eventData.lastPress && button.pressTime - eventData.clickTime <= kClickSpeed;
                if (eventData.clickCount > 0 && !button.clickedOnSameGameObject)
                {
                    eventData.clickCount = default;
                    eventData.clickTime = default;
                }

                // Set pointerPress. This nukes lastPress. Meaning that after
OnPointerDown, lastPress will
                // become null.
                eventData.pointerPress = newPressed;
#if UNITY_2020_1_OR_NEWER // pointerClick doesn't exist before this.
                eventData.pointerClick = pointerClickHandler;
#endif
                eventData.rawPointerPress = currentOverGo;

                // Save the drag handler for drag events during this mouse down.
                eventData.pointerDrag =
ExecuteEvents.GetEventHandler<IDragHandler>(currentOverGo);

                if (eventData.pointerDrag != null)
                    ExecuteEvents.Execute(eventData.pointerDrag, eventData,
ExecuteEvents.initializePotentialDrag);
            }

            // Button release.
            if (button.wasReleasedThisFrame)
            {
                // Check for click. Release must be on same GO that we pressed on
and we must not
                // have moved beyond our move tolerance (doing so will set
eligibleForClick to false).
                // NOTE: There's two difference to click handling here compared
to StandaloneInputModule.
                //       1) StandaloneInputModule counts clicks entirely on press
meaning that clickCount is increased
                //          before a click has actually happened.
                //       2) StandaloneInputModule increases click counts even if
something is eventually not deemed a
                //          click and OnPointerClick is thus never invoked.
                var pointerClickHandler =
ExecuteEvents.GetEventHandler<IPointerClickHandler>(currentOverGo);
#if UNITY_2020_1_OR_NEWER
                var isClick = eventData.pointerClick != null &&
eventData.pointerClick == pointerClickHandler && eventData.eligibleForClick;
```

```
#else
                var isClick = eventData.pointerPress != null &&
eventData.pointerPress == pointerClickHandler && eventData.eligibleForClick;
#endif
                if (isClick)
                {
                    // Count clicks.
                    if (button.clickedOnSameGameObject)
                    {
                        // We re-clicked on the same UI element within 0.3
seconds so count
                        // it as a repeat click.
                        ++eventData.clickCount;
                    }
                    else
                    {
                        // First click on this object.
                        eventData.clickCount = 1;
                    }
                    eventData.clickTime =
InputRuntime.s_Instance.unscaledGameTime;
                }

                // Invoke OnPointerUp.
                ExecuteEvents.Execute(eventData.pointerPress, eventData,
ExecuteEvents.pointerUpHandler);

                // Invoke OnPointerClick or OnDrop.
                if (isClick)
                {
#if UNITY_2020_1_OR_NEWER
                    ExecuteEvents.Execute(eventData.pointerClick, eventData,
ExecuteEvents.pointerClickHandler);
#else
                    ExecuteEvents.Execute(eventData.pointerPress, eventData,
ExecuteEvents.pointerClickHandler);
#endif
                }
                else if (eventData.dragging && eventData.pointerDrag != null)
                    ExecuteEvents.ExecuteHierarchy(currentOverGo, eventData,
ExecuteEvents.dropHandler);

                eventData.eligibleForClick = false;
                eventData.pointerPress = null;
                eventData.rawPointerPress = null;

                if (eventData.dragging && eventData.pointerDrag != null)
                    ExecuteEvents.Execute(eventData.pointerDrag, eventData,
ExecuteEvents.endDragHandler);

                eventData.dragging = false;
                eventData.pointerDrag = null;

                button.ignoreNextClick = false;
            }

            button.CopyPressStateFrom(eventData);
        }
```

- Button:OnPointerClick

```
public virtual void OnPointerClick(PointerEventData eventData)
{
    if (eventData.button != PointerEventData.InputButton.Left)
        return;

    Press();
}
```

- Button:Press

```
private void Press()
{
    if (!IsActive() || !IsInteractable())
        return;

    UISystemProfilerApi.AddMarker("Button.onClick", this);
    m_OnClick.Invoke();
}
```

到此一次完整点击按钮事件完成

总结一下：Unity EventSystem每一帧都在Tick Input模块输入 -> InputSystemUIInputModule处理输入：更新point（鼠标，触屏点）状态，更新hover ui elements，鼠标点击/释放->OnPointerClick->Press

看起来很简单

- Unity每一帧都在进行输入更新和射线检测，不管鼠标有没有移动，因为有OnPointerEnter/Exit 对应事件
- Unity按下按钮触发实际上是在鼠标松开的时候，如果一直按着移出去是不会触发点击事件
- Unity触发事件实际上是链式触发，对于Button点击，实际上是click了TMP，TMP没有相应click事件所以输入模块向上查找能处理事件的对象，找到 Button。

为什么点击是TMP?

我们需要回到射线检测的地方looklook

在ProcessPointer中`eventData.pointerCurrentRaycast = PerformRaycast(eventData);`获得点击的GO

```
        private RaycastResult PerformRaycast(ExtendedPointerEventData eventData)
        {
            if (eventData == null)
                throw new ArgumentNullException(nameof(eventData));

            .......

            // Otherwise pass it along to the normal raycasting logic.
            eventSystem.RaycastAll(eventData, m_RaycastResultCache);
            // 这一行就是找到第一个合法的GO，忽略也没问题
            var result = FindFirstRaycast(m_RaycastResultCache);
            m_RaycastResultCache.Clear();
            return result;
```

```
        }

        public void RaycastAll(PointerEventData eventData, List<RaycastResult>
raycastResults)
        {
            raycastResults.Clear();
            var modules = RaycasterManager.GetRaycasters();
            var modulesCount = modules.Count;
            for (int i = 0; i < modulesCount; ++i)
            {
                var module = modules[i];
                if (module == null || !module.IsActive())
                    continue;

                module.Raycast(eventData, raycastResults);
            }

            raycastResults.Sort(s_RaycastComparer);
        }
        private static int RaycastComparer(RaycastResult lhs, RaycastResult rhs)
        {
            if (lhs.module != rhs.module)
            {
                var lhsEventCamera = lhs.module.eventCamera;
                var rhsEventCamera = rhs.module.eventCamera;
                if (lhsEventCamera != null && rhsEventCamera != null &&
lhsEventCamera.depth != rhsEventCamera.depth)
                {
                    // need to reverse the standard compareTo
                    if (lhsEventCamera.depth < rhsEventCamera.depth)
                        return 1;
                    if (lhsEventCamera.depth == rhsEventCamera.depth)
                        return 0;

                    return -1;
                }

                if (lhs.module.sortOrderPriority !=
rhs.module.sortOrderPriority)
                    return
rhs.module.sortOrderPriority.CompareTo(lhs.module.sortOrderPriority);

                if (lhs.module.renderOrderPriority !=
rhs.module.renderOrderPriority)
                    return
rhs.module.renderOrderPriority.CompareTo(lhs.module.renderOrderPriority);
            }

            // Renderer sorting
            if (lhs.sortingLayer != rhs.sortingLayer)
            {
                // Uses the layer value to properly compare the relative order of
the layers.
                var rid = SortingLayer.GetLayerValueFromID(rhs.sortingLayer);
                var lid = SortingLayer.GetLayerValueFromID(lhs.sortingLayer);
                return rid.CompareTo(lid);
            }
```

```
            if (lhs.sortingOrder != rhs.sortingOrder)
                return rhs.sortingOrder.CompareTo(lhs.sortingOrder);

            // comparing depth only makes sense if the two raycast results have
the same root canvas (case 912396)
            if (lhs.depth != rhs.depth && lhs.module.rootRaycaster ==
rhs.module.rootRaycaster)
                return rhs.depth.CompareTo(lhs.depth);

            if (lhs.distance != rhs.distance)
                return lhs.distance.CompareTo(rhs.distance);

            #if PACKAGE_PHYSICS2D
            // Sorting group
            if (lhs.sortingGroupID != SortingGroup.invalidSortingGroupID &&
rhs.sortingGroupID != SortingGroup.invalidSortingGroupID)
            {
                if (lhs.sortingGroupID != rhs.sortingGroupID)
                    return lhs.sortingGroupID.CompareTo(rhs.sortingGroupID);
                if (lhs.sortingGroupOrder != rhs.sortingGroupOrder)
                    return
rhs.sortingGroupOrder.CompareTo(lhs.sortingGroupOrder);
            }
            #endif

            return lhs.index.CompareTo(rhs.index);
        }
```

排序逻辑

- 比较Raycaster，是不是一个同一个射线检测到的物体？
  - 不是

    > 首先比较相机depth，因为射线从射线检测器，射线总有一个发送点，一般是相机发送的嘛（But相机从哪里来？）
    > 然后比较sortOrderPriority
    > 继续比较renderOrderPriority

  - 是 or 以上比较都相等，进入这一次比较

    > 比较sortingLayer
    >
    > 比较sortingOrder
    >
    > depth比较（for UI in same canvas 啥是depth？就是ui渲染的深度值，depth越大的越后渲染）
    >
    > distance比较（ for 物理检测，距离近优先）

以上了解了，也就知道为啥其实是TMP了，TMP在button之后渲染，depth更大，所以是TMP

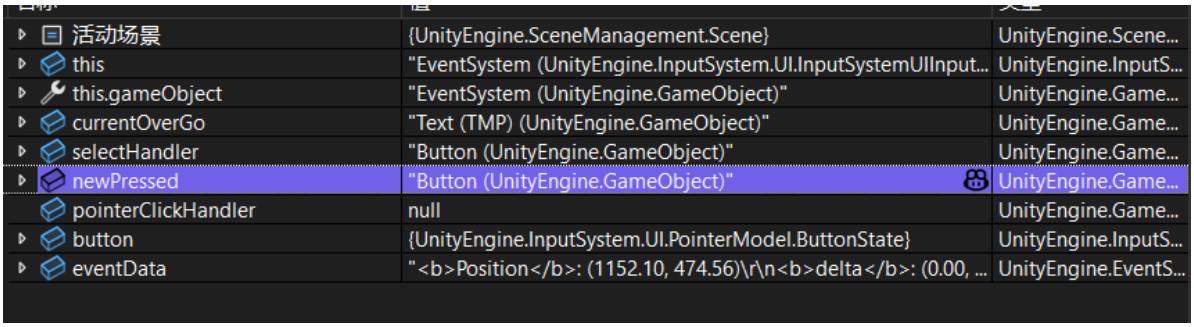
（关于depth怎么排的，为什么TMP更大？这就涉及到UGUI渲染部分了，在此pass，感兴趣自己学:））

相机来源:

Raycaster 类型	通常挂在哪里	检测对象	相机来源
`GraphicRaycaster`	Canvas 上	UI 图形	Canvas 的相机
`PhysicsRaycaster`	Camera 上	3D Collider	Camera
`Physics2DRaycaster`	Camera 上	2D Collider	Camera

层级转发？ and 移除不点击?

在ProcessPointerButton中




```
        private void ProcessPointerButton(ref PointerModel.ButtonState button,
PointerEventData eventData)
        {
            var currentOverGo = eventData.pointerCurrentRaycast.gameObject;

            if (currentOverGo != null &&
PointerShouldIgnoreTransform(currentOverGo.transform))
                return;

            // Button press.
            if (button.wasPressedThisFrame)
            {
                button.pressTime = InputRuntime.s_Instance.unscaledGameTime;

                eventData.delta = Vector2.zero;
                eventData.dragging = false;
                eventData.pressPosition = eventData.position;
                eventData.pointerPressRaycast = eventData.pointerCurrentRaycast;
                eventData.eligibleForClick = true;
                eventData.useDragThreshold = true;

                var selectHandler =
ExecuteEvents.GetEventHandler<ISelectHandler>(currentOverGo);

                // If we have clicked something new, deselect the old thing and
leave 'selection handling' up
                // to the press event (except if there's none and we're told to
not deselect in that case).
                if (selectHandler != eventSystem.currentSelectedGameObject &&
(selectHandler != null || m_DeselectOnBackgroundClick))
                    eventSystem.SetSelectedGameObject(null, eventData);

                // Invoke OnPointerDown, if present.
                var newPressed = ExecuteEvents.ExecuteHierarchy(currentOverGo,
eventData, ExecuteEvents.pointerDownHandler);
```

```
                var pointerClickHandler =
ExecuteEvents.GetEventHandler<IPointerClickHandler>(currentOverGo);

                // If no GO responded to OnPointerDown, look for one that
responds to OnPointerClick.
                // NOTE: This only looks up the handler. We don't invoke
OnPointerClick here.
                if (newPressed == null)
                    newPressed = pointerClickHandler;

                // Reset click state if delay to last release was too long or if
we didn't
                // press on the same object as last time. The latter part we
don't know until
                // we've actually run the press handler.
                button.clickedOnSameGameObject = newPressed ==
eventData.lastPress && button.pressTime - eventData.clickTime <= kClickSpeed;
                if (eventData.clickCount > 0 && !button.clickedOnSameGameObject)
                {
                    eventData.clickCount = default;
                    eventData.clickTime = default;
                }

                // Set pointerPress. This nukes lastPress. Meaning that after
OnPointerDown, lastPress will
                // become null.
                eventData.pointerPress = newPressed;
                // 保存按下时点击物体
#if UNITY_2020_1_OR_NEWER // pointerClick doesn't exist before this.
                eventData.pointerClick = pointerClickHandler;
#endif
                eventData.rawPointerPress = currentOverGo;

                // Save the drag handler for drag events during this mouse down.
                eventData.pointerDrag =
ExecuteEvents.GetEventHandler<IDragHandler>(currentOverGo);

                if (eventData.pointerDrag != null)
                    ExecuteEvents.Execute(eventData.pointerDrag, eventData,
ExecuteEvents.initializePotentialDrag);
            }

            // Button release.
            if (button.wasReleasedThisFrame)
            {
                // Check for click. Release must be on same GO that we pressed on
and we must not
                // have moved beyond our move tolerance (doing so will set
eligibleForClick to false).
                // NOTE: There's two difference to click handling here compared
to StandaloneInputModule.
                //       1) StandaloneInputModule counts clicks entirely on press
meaning that clickCount is increased
                //          before a click has actually happened.
                //       2) StandaloneInputModule increases click counts even if
something is eventually not deemed a
                //          click and OnPointerClick is thus never invoked.
```

```
                // 找到当前鼠标所在物体的点击接受对象
                var pointerClickHandler =
ExecuteEvents.GetEventHandler<IPointerClickHandler>(currentOverGo);
                // 比较和上一次是不是同一个
#if UNITY_2020_1_OR_NEWER
                var isClick = eventData.pointerClick != null &&
eventData.pointerClick == pointerClickHandler && eventData.eligibleForClick;
#else
                var isClick = eventData.pointerPress != null &&
eventData.pointerPress == pointerClickHandler && eventData.eligibleForClick;
#endif
                if (isClick)
                {
                    // Count clicks.
                    if (button.clickedOnSameGameObject)
                    {
                        // We re-clicked on the same UI element within 0.3
seconds so count
                        // it as a repeat click.
                        ++eventData.clickCount;
                    }
                    else
                    {
                        // First click on this object.
                        eventData.clickCount = 1;
                    }
                    eventData.clickTime =
InputRuntime.s_Instance.unscaledGameTime;
                }

                // Invoke OnPointerUp.
                ExecuteEvents.Execute(eventData.pointerPress, eventData,
ExecuteEvents.pointerUpHandler);

                // Invoke OnPointerClick or OnDrop.
                if (isClick)
                {
#if UNITY_2020_1_OR_NEWER
                    ExecuteEvents.Execute(eventData.pointerClick, eventData,
ExecuteEvents.pointerClickHandler);
#else
                    ExecuteEvents.Execute(eventData.pointerPress, eventData,
ExecuteEvents.pointerClickHandler);
#endif
                }
                else if (eventData.dragging && eventData.pointerDrag != null)
                    ExecuteEvents.ExecuteHierarchy(currentOverGo, eventData,
ExecuteEvents.dropHandler);

                eventData.eligibleForClick = false;
                eventData.pointerPress = null;
                eventData.rawPointerPress = null;

                if (eventData.dragging && eventData.pointerDrag != null)
                    ExecuteEvents.Execute(eventData.pointerDrag, eventData,
ExecuteEvents.endDragHandler);

                eventData.dragging = false;
```

```
                eventData.pointerDrag = null;

                button.ignoreNextClick = false;
            }

            button.CopyPressStateFrom(eventData);
        }
```

总结：处理点击事件时会根据当前射线物体一直向上找到第一个接受对应事件的GO，处理对应时间，且会比较鼠标按下时和松开时是否时同一个处理的物体。

怎么向上找到接受对应处理的物体

一路寻找满足条件的parent。

GetEventHandler+Execute基本等价ExecuteHierarchy。

```
        public static GameObject GetEventHandler<T>(GameObject root) where T :
IEventSystemHandler
        {
            if (root == null)
                return null;

            Transform t = root.transform;
            while (t != null)
            {
                if (CanHandleEvent<T>(t.gameObject))
                    return t.gameObject;
                t = t.parent;
            }
            return null;
        }

        public static bool Execute<T>(GameObject target, BaseEventData
eventData, EventFunction<T> functor) where T : IEventSystemHandler
        {
            var internalHandlers = ListPool<IEventSystemHandler>.Get();
            GetEventList<T>(target, internalHandlers);
            //  if (s_InternalHandlers.Count > 0)
            //      Debug.Log("Executinng " + typeof (T) + " on " + target);

            var internalHandlersCount = internalHandlers.Count;
            for (var i = 0; i < internalHandlersCount; i++)
            {
                T arg;
                try
                {
                    arg = (T)internalHandlers[i];
                }
                catch (Exception e)
                {
                    var temp = internalHandlers[i];
                    Debug.LogException(new Exception(string.Format("Type {0}
expected {1} received.", typeof(T).Name, temp.GetType().Name), e));
                    continue;
                }
```

```
                try
                {
                    functor(arg, eventData);
                }
                catch (Exception e)
                {
                    Debug.LogException(e);
                }
            }

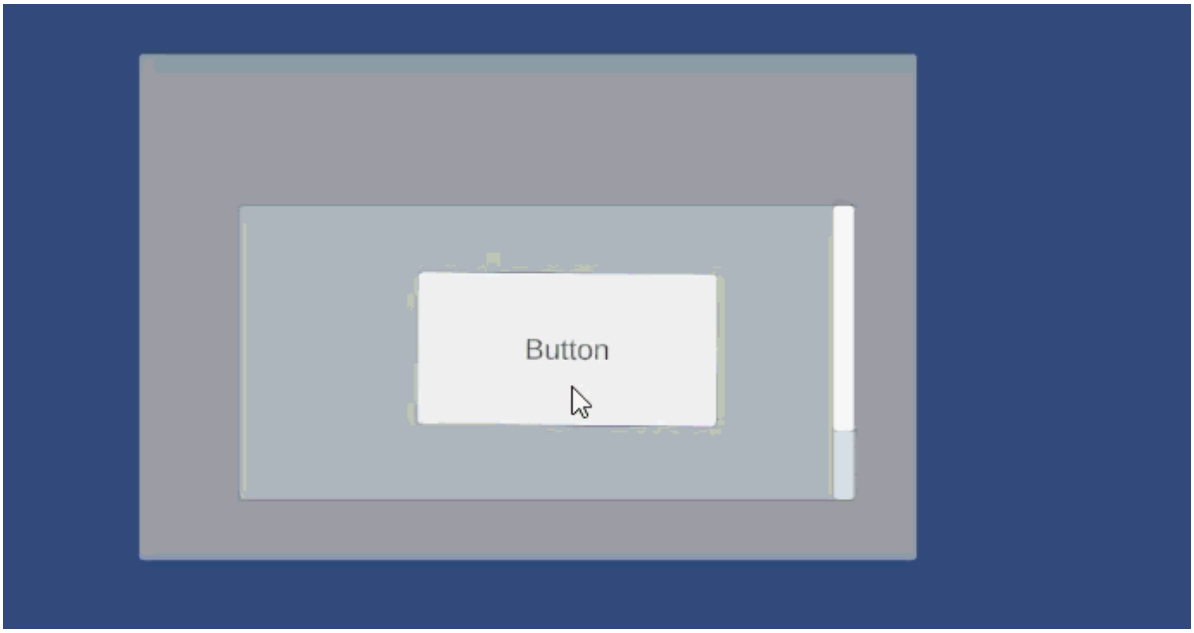
            var handlerCount = internalHandlers.Count;
            ListPool<IEventSystemHandler>.Release(internalHandlers);
            return handlerCount > 0;
        }

        public static GameObject ExecuteHierarchy<T>(GameObject root,
BaseEventData eventData, EventFunction<T> callbackFunction) where T :
IEventSystemHandler
        {
            GetEventChain(root, s_InternalTransformList);

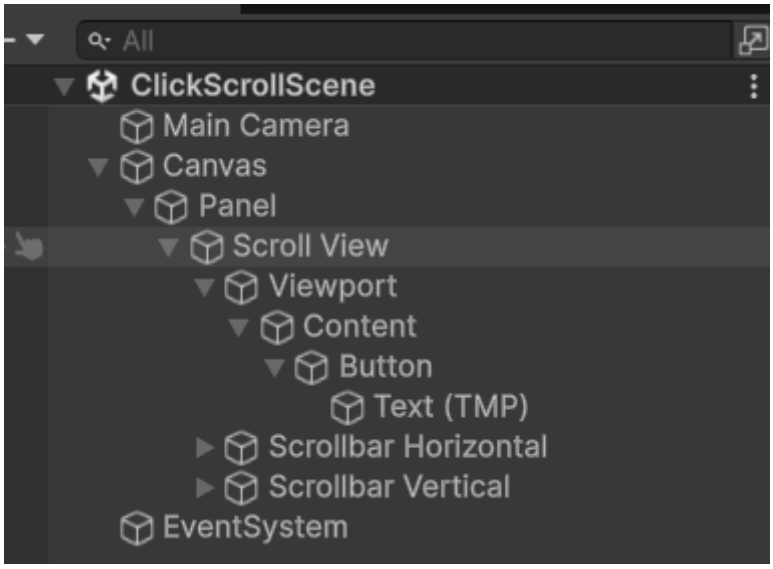
            var internalTransformListCount = s_InternalTransformList.Count;
            for (var i = 0; i < internalTransformListCount; i++)
            {
                var transform = s_InternalTransformList[i];
                if (Execute(transform.gameObject, eventData, callbackFunction))
                    return transform.gameObject;
            }
            return null;
        }
        private static void GetEventChain(GameObject root, IList<Transform>
eventChain)
        {
            eventChain.Clear();
            if (root == null)
                return;

            var t = root.transform;
            while (t != null)
            {
                eventChain.Add(t);
                t = t.parent;
            }
        }
```

Scroll and Click?



https://cdn.jsdelivr.net/gh/forestlyn/Drawing-bed/blog/202610221233040.gif



看图，虽然按住了Buntton但是拖拽时Scroll移动，且松开没触发Button Click。单独点击一次才出发了Click，Why?

- 在ProcessPointerButton中，按下我们处理了潜在的拖拽事件，没有实际真的开始Drag!

```
eventData.pointerDrag =
    ExecuteEvents.GetEventHandler<IDragHandler>(currentOverGo);

if (eventData.pointerDrag != null)
    ExecuteEvents.Execute(
        eventData.pointerDrag,
        eventData,
        ExecuteEvents.initializePotentialDrag);
```

- ProcessPointerButtonDrag处理拖拽

  指针没移动，跳过。鼠标 / 笔且光标锁定，跳过。没找到拖拽目标，跳过。

  开始处理拖拽。

  拖拽阈值判断，移动超过阈值认定在拖拽，**将点击事件取消**。不是直接取消但是将对应条件置空/false，因此不会触发click。

`var isClick = eventData.pointerClick != null && eventData.pointerClick == pointerClickHandler && eventData.eligibleForClick;`

```
private void ProcessPointerButtonDrag(ref PointerModel.ButtonState button, ExtendedPointerEventData eventData)
{
    if (!eventData.IsPointerMoving() ||
        (eventData.pointerType == UIPointerType.MouseOrPen && Cursor.lockState == CursorLockMode.Locked) ||
        eventData.pointerDrag == null)
        return;

    // Detect drags.
    if (!eventData.dragging)
    {
        if (!eventData.useDragThreshold || (eventData.pressPosition - eventData.position).sqrMagnitude >=
            (double)eventSystem.pixelDragThreshold * eventSystem.pixelDragThreshold * (eventData.pointerType == UIPointerType.Tracked

      ? m_TrackedDeviceDragThresholdMultiplier

      : 1))
        {
            // Started dragging. Invoke OnBeginDrag.
            ExecuteEvents.Execute(eventData.pointerDrag, eventData, ExecuteEvents.beginDragHandler);
            eventData.dragging = true;
        }
    }

    if (eventData.dragging)
    {
        // If we moved from our initial press object, process an up for that object.
        if (eventData.pointerPress != eventData.pointerDrag)
        {
            ExecuteEvents.Execute(eventData.pointerPress, eventData, ExecuteEvents.pointerUpHandler);

            eventData.eligibleForClick = false;
            eventData.pointerPress = null;
            eventData.rawPointerPress = null;
        }

        ExecuteEvents.Execute(eventData.pointerDrag, eventData, ExecuteEvents.dragHandler);
        button.CopyPressStateFrom(eventData);
    }
}
```

总结

按钮点击、列表拖拽，其实都是同一个事件链路：

角色	主要职责
`EventSystem`	管理选中对象，调度输入模块
`InputSystemUIInputModule`	处理输入状态，判断点击、拖拽等交互
`Raycaster`	找到符合条件的命中对象
`ExecuteEvents`	查找事件处理组件并调用接口
`Button`、`ScrollRect` 等组件	响应事件，执行控件行为

理解这条链路后，遇到交互异常，可以按三个问题检查：

1. **命中了谁？**
   查看 `pointerCurrentRaycast`，检查遮挡、Raycast Target 和射线过滤条件。
2. **事件交给了谁？**
   查看 `pointerPress`、`pointerClick`、`pointerDrag`，检查父子节点上的事件接口。
3. **为什么执行或没有执行？**
   检查 `eligibleForClick`、拖拽状态，以及控件的可交互状态。

参考

Event System | Unity UI | 2.0.0

UGUI源码解析——点击事件